Population shifts in Finland

LAB University of Applied Sciences
Bachelor's Degree Programme in Industrial Information Technology
2026

LAB University of Applied Sciences

Roland Spielmann

Statement about using AI

The thesis must always include a statement regarding the author's responsibilities and any use of AI, such as:

Example 1:

The author of this thesis, John Doe, is responsible for the accuracy of the entire contents of the thesis and has not used AI in writing it.

The authenticity of this thesis has been checked with the Turnitin similarity checking software.

Example 2:

The author of this thesis, John Doe, is responsible for the accuracy of the entire contents of the thesis, including content generated with AI. AI was used in this work during background research and later for language editing.

Background research made use of Copilot (Microsoft Copilot: Your everyday AI companion).

The following prompts were entered on August 28, 2025:

How does a heat pump work?

The generative AI output was adapted and edited for the final answer. Original output has been saved and is available upon request.

DeepL (DeepL Translate: The world’s most accurate translator) was used for language checking to ensure linguistic correctness. The use was occasional throughout the thesis.

The authenticity of this thesis has been checked with the Turnitin similarity checking software.

Abstract

Author(s)	Publication type	Completion year
Rolands Prohorovs	Thesis	2026
	Number of pages xx	

Title of the thesis **Population shifts in Finland**
Degree, Field of Study Bachelor's Degree Programme in Industrial Information Technology
Name, title and organisation of the client Lab University of Applied Science
Abstract
Keywords

Contents

1 Data Analysis 5
2 Building Model 9
3 Training Model on local machine vs supercomputer 11
4 References 18

Appendix 1. Appendix title

Appendix 2. Appendix title

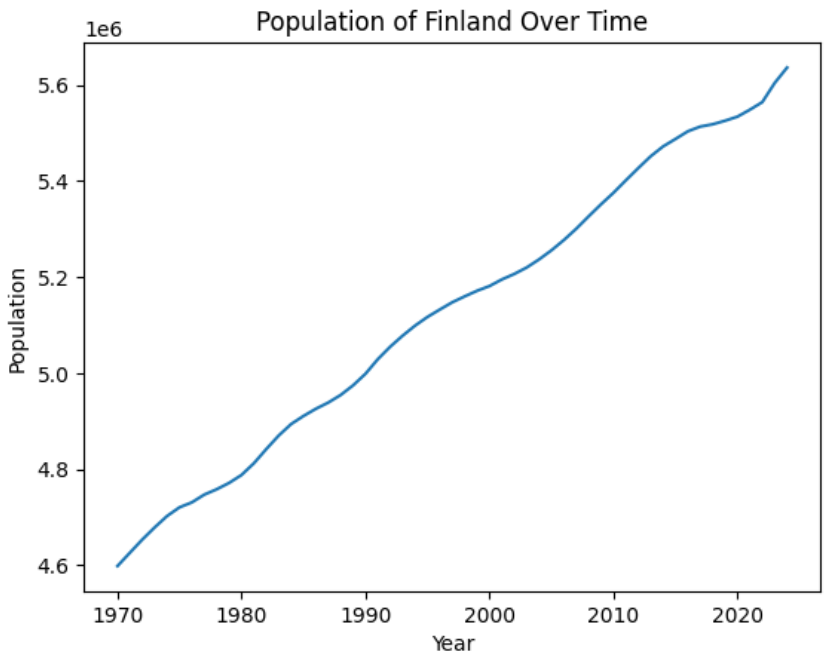
1 Data Analysis

I have looked through a lot of different datasets. I wanted to use one big dataset and not a lot of small ones. In the end I have picked dataset that contains these columns:

1. Year
2. Live
3. Births
4. Deaths
5. Natural increase
6. Intermunicipal
7. Migration
8. Immigration to Finland
9. Emigration from Finland
10. Net migration
11. Marriages
12. Divorces
13. Total change
14. Population

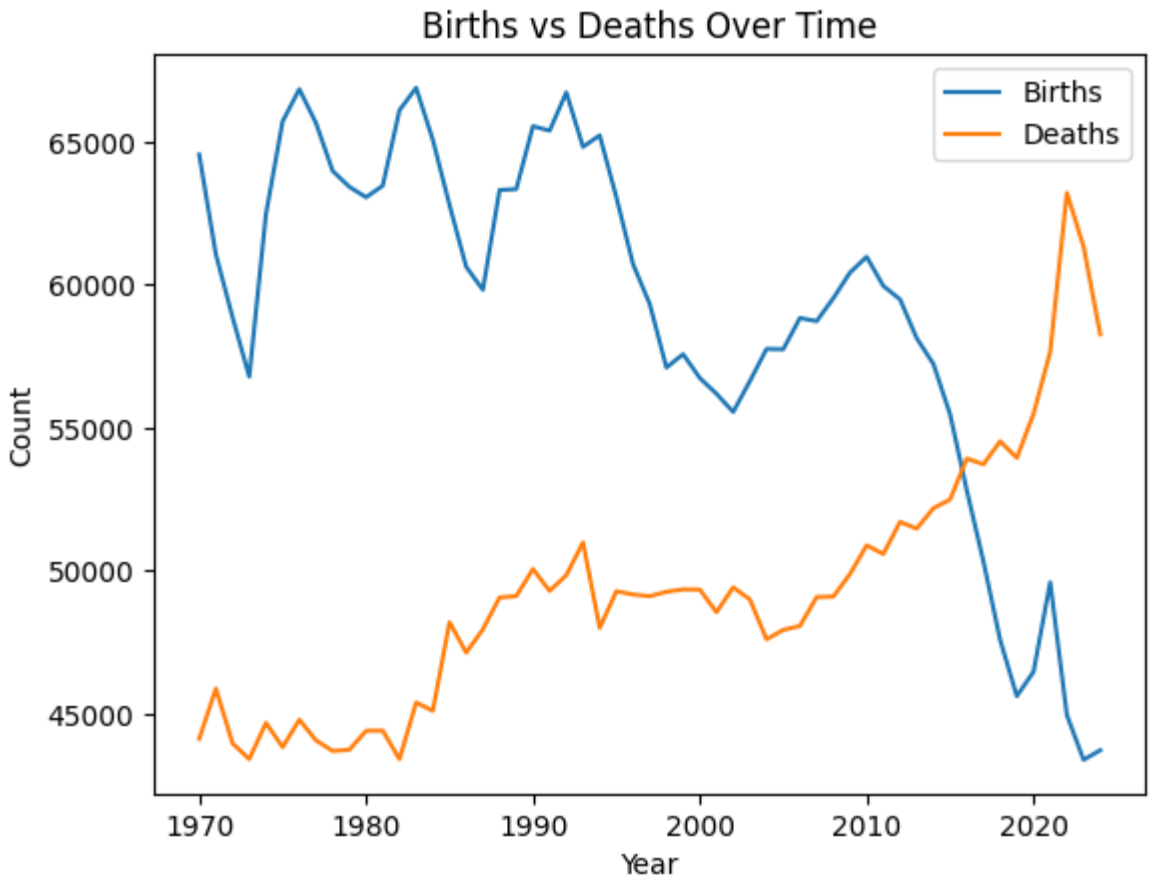
I liked these one because it contains different columns that would be useful to compare like natural increase and net migration or births and deaths.

- First graph: **Population of Finland Over Time:**



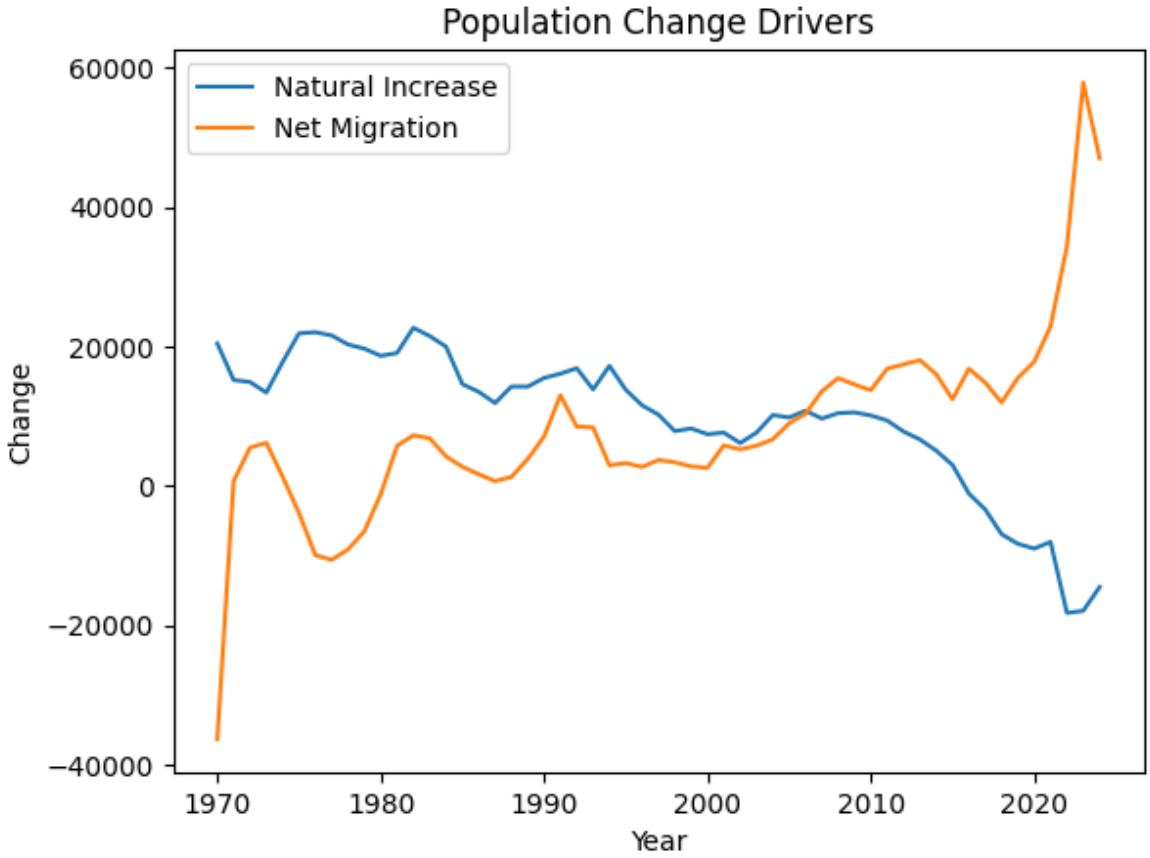
As instructed, I have used years since 1970 to 2024. There is not much to analyse only that Finnish population was increasing very evenly for these 54 years. I have also calculated that population grew by 1 037 635 people, which is ~22.6%.

- Second graph: **Births vs Death Over Time:**



The graph shows that since 1970 till approximately 1990 number of births was high. In the approximately 1990 number of births started to decrease dramatically. In ~2018 lines crossed and since then number of deaths is way higher that births, which means that natural increase is not main driver of population growth. This also mean that migration is important in growing total population.

- Third graph: **Population Change Drivers:**



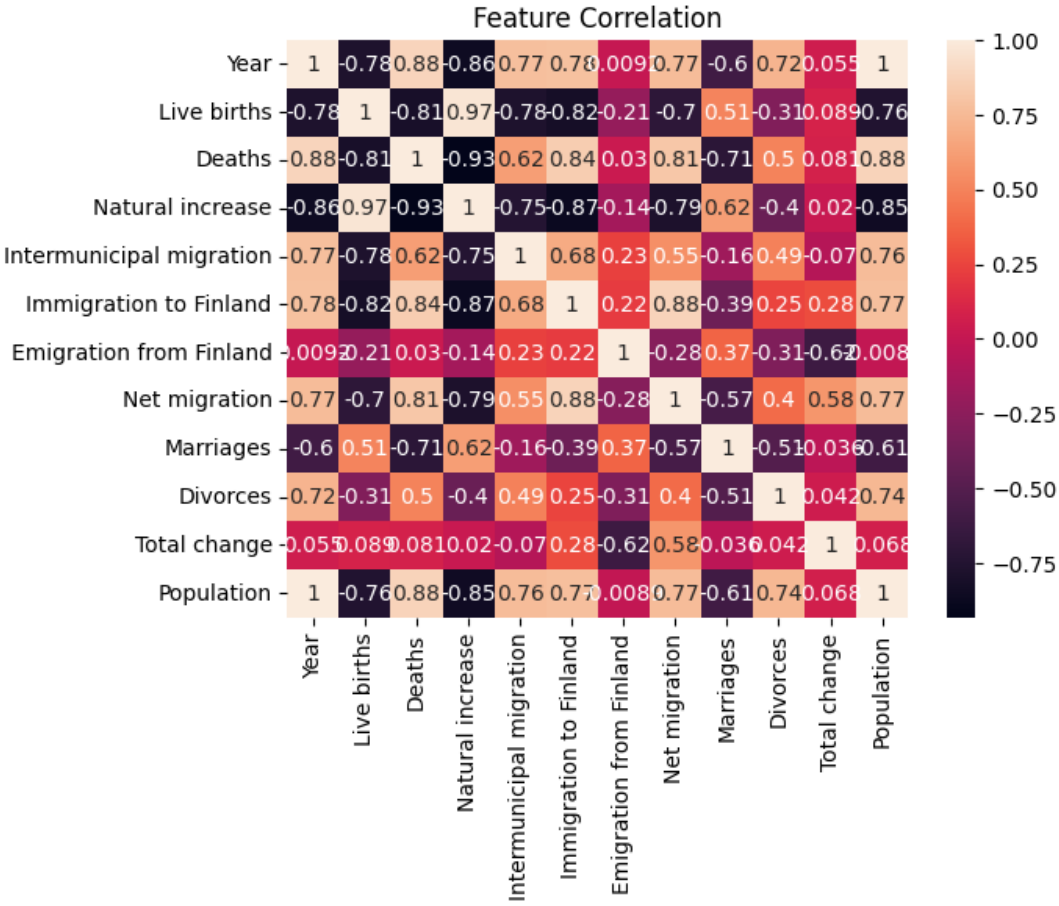
The graph shows two main drivers of population change in Finland: natural increase and net migration.

In the earlier decades, natural increase was clearly the dominant factor in population growth, being consistently positive and high. During this period, net migration was often negative, so more people were leaving the country than entering.

Starting from the 1990 and especially after 2000, natural increase started declining. This reflects decreasing birth rates and an aging population. At the same time, net migration became positive.

A turning point was in the ~2008, when net migration became higher than natural increase. In ~2015 natural increase even becomes negative. But population continues to grow due to positive net migration.

- Fourth graph: **Feature Correlation:**



This graph is most important one, because on it I can see what parrameters in the dataset really correlate the most, and which ones only look like it.

Correlation values range from -1 to 1, where values close to 1 indicate a strong positive relationship, values close to -1 indicate a strong negative relationship.

Natural increase shows a strong negative correlation with deaths and a strong positive correlation with live births, which is expected since it is calculated as the difference between births and deaths.

Net migration has a strong positive correlation with immigration and a negative correlation with emigration, which means it plays key role in driving a population change. It also shows a positive relationship with total population, meaning that migration contributes significantly to population growth.

Additionally, the correlation between year and several variables highlights long-term trends, including population aging and demographic transition.

2 Building Model

I have built model in the Jupiter notebook. In the end I got 6 different versions of a model. They all are similar but use different parameters and some are more advanced.
Here I want to describe model number 5. later I will train it on the supercomputer to compare.

System that I have used to make forecasting combines two components:

- A **neural network** that predicts population change.
- An **age structured cohort model** that learns how population ages and shows demographic features to the network.

I will describe neural network.

It uses 5 layers:

- **Input layer:** 11 features (year, migration, natural increase, lagged population values, age-group sizes, decency ration)
- **Hidden layers:** three Dense layers with 64, 32 and 16 neurons, ReLu activation
- **Output layer:** single neuron with linear activation

Key parameters:

- **Epochs:** 200
- **Learning rate:** 0.005
- **Batch size:** 32
- **Optimizer:** Adam
- **Loss function:** mean squared error
- **Metric:** mean absolute error
- **Early stopping:** patience 20 epochs

How it works:

1. **Input scaling:** features and target are standardised to zero mean and unit variance.
2. **Forward pass:** 11 scaled inputs pass through hidden layers. The output layer produces a scaled delta prediction.
3. **Loss and backpropagation:** MSE compares predicted and true delta. Optimiser updates weights.
4. **Training loop:** up to 200 epochs, batch size 32. Early stopping restores best weights after 20 epochs. Usually training stops at 70 –80 epochs
5. **Prediction:** scaled delta is inverse-transformed and added to previous year's population. The age structured model updates age distribution, giving new features into the network for the next forecast.

Why it works:

Neural network instead of a simple trend – population change deepens non-linearly on migration, natural increase and age structure. A small dense network sees these connections better than a linear model. According to ScienceDirect n.d. V. Riiman, A. Wilson, R. Milewicz, P. Pirkelbauer LSTM networks trained only on population totals outperformed traditional cohort-component models (MAPE: 5.0% vs 6.5%) for Alabama county forecasts, despite requiring less detailed input data.

Predicting deltas not absolute population – the network learns the change, which is more stationary and easier to extrapolate than the raw population series. ScienceDirect n.d. V. Riiman, A. Wilson, R. Milewicz, P. Pirkelbauer similarly found that neural networks can produce accurate forecasts even with minimal feature engineering when the target variable is properly structured.

3 Training Model on local machine vs supercomputer

I have successfully logged into puhti supercomputer and started new Jupiter notebook session.

Error:

I have a lot of errors due to TensorFlow. I couldn't install it directly through terminal and premade version also haven't worked for me. For that reason, I have restarted session couple times. Solution was to change python module.

Session characteristics:

Partition: gpu

Core: 6

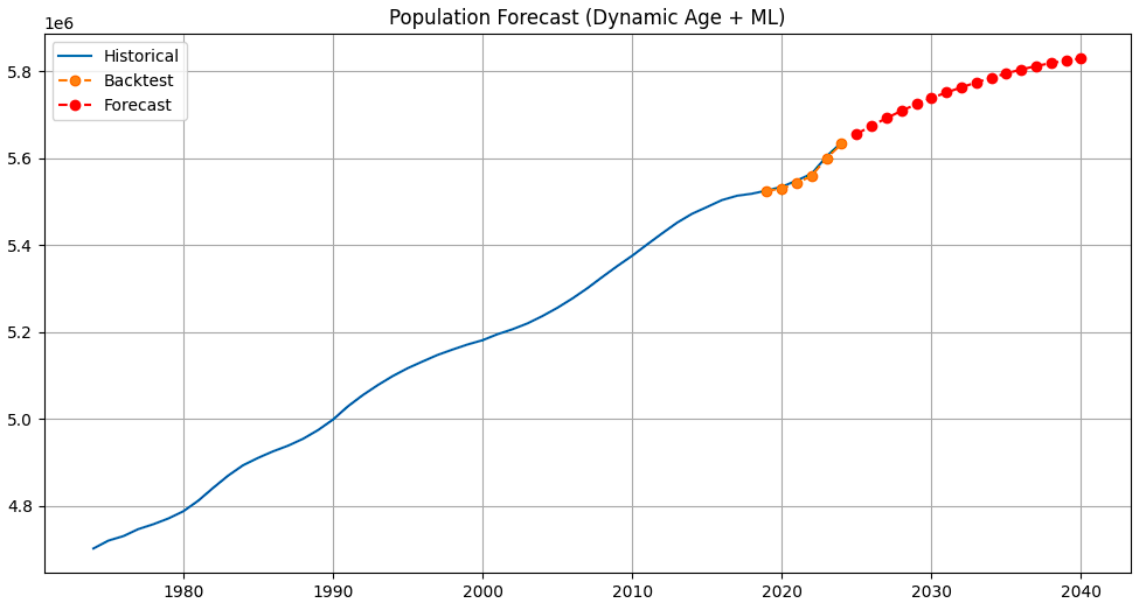
Memory: 24

Python module: TensorFlow

Time: 4 hours

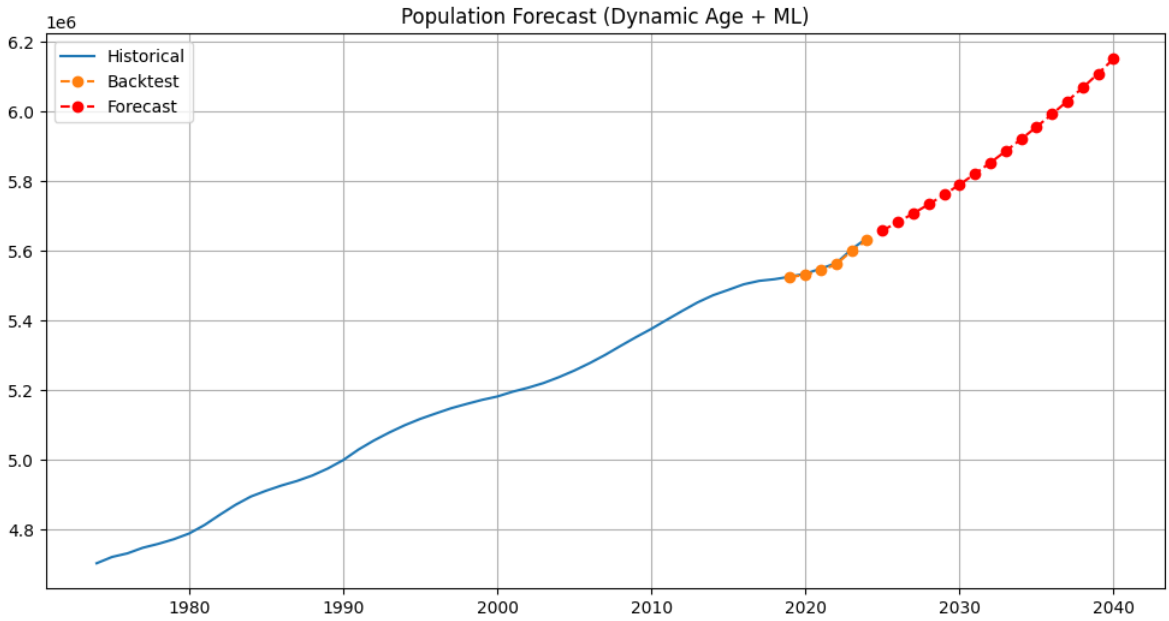
Training model:

Firstly, I have tried model that gave me best results. I have noticed that speed is approximately two times faster than on my local machine. I have 200 epoch in this model and training completed in second. And I got these results:



```
FORECAST 2025-2040
2025: 5,655,261
2026: 5,673,972
2027: 5,691,717
2028: 5,708,462
2029: 5,723,881
2030: 5,737,903
2031: 5,750,490
2032: 5,762,385
2033: 5,773,713
2034: 5,784,410
2035: 5,794,481
2036: 5,803,518
2037: 5,811,420
2038: 5,818,191
2039: 5,823,966
2040: 5,828,759
```

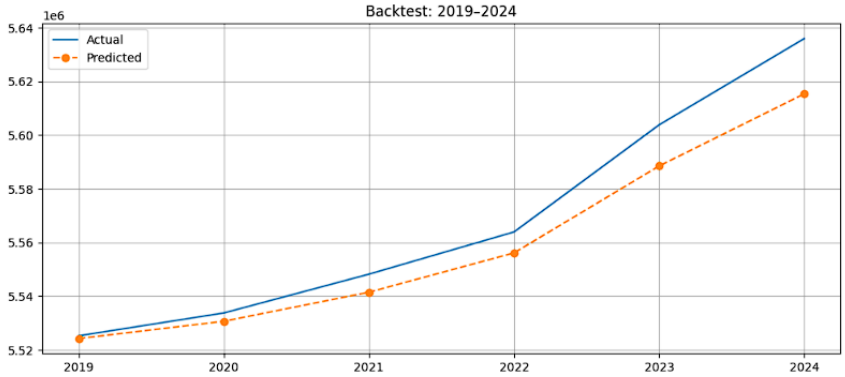
On the first look I have thought that results are the same, but then I noticed that prediction line has more curve. Comparing to the results of the model that is trained on local machine:

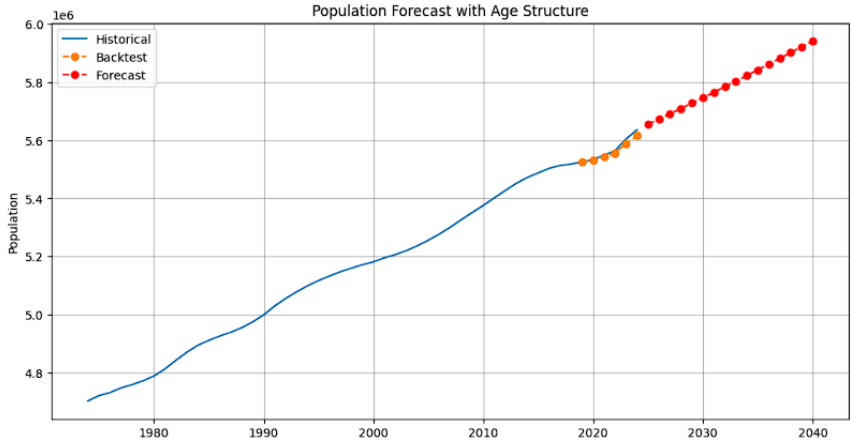


This model takes in account most features like year, migration, natural increase, age of population, aging of population. That can be reason why model learned differently.

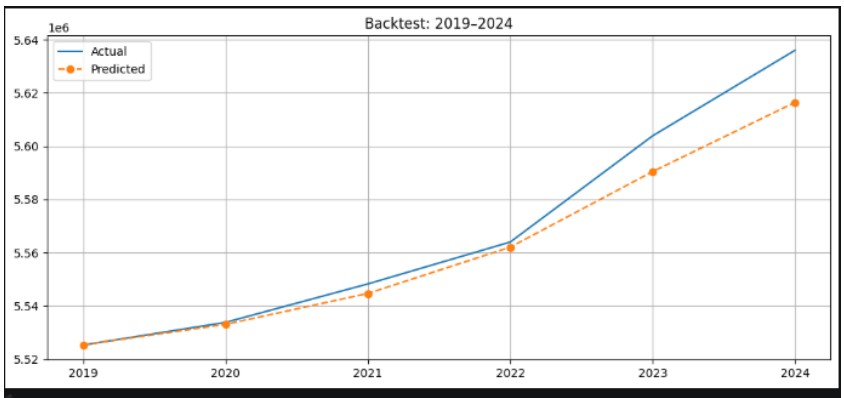
Then I have tried another model that has separate chart for backrest:

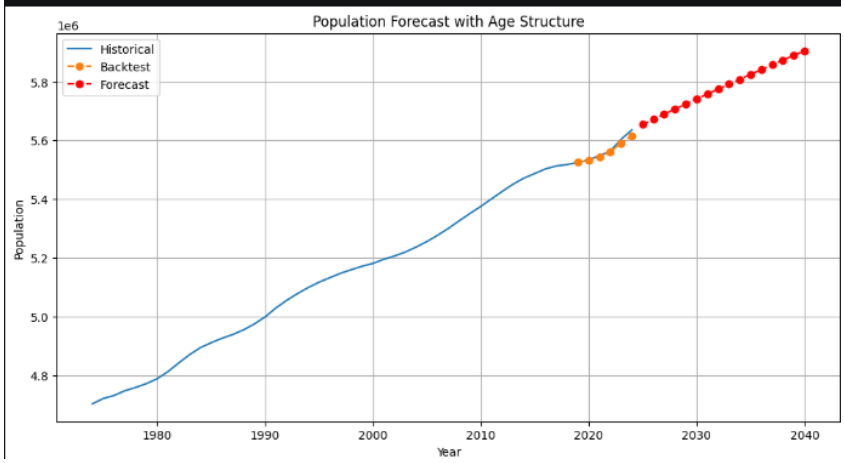
Supercomputer:





Local machine:

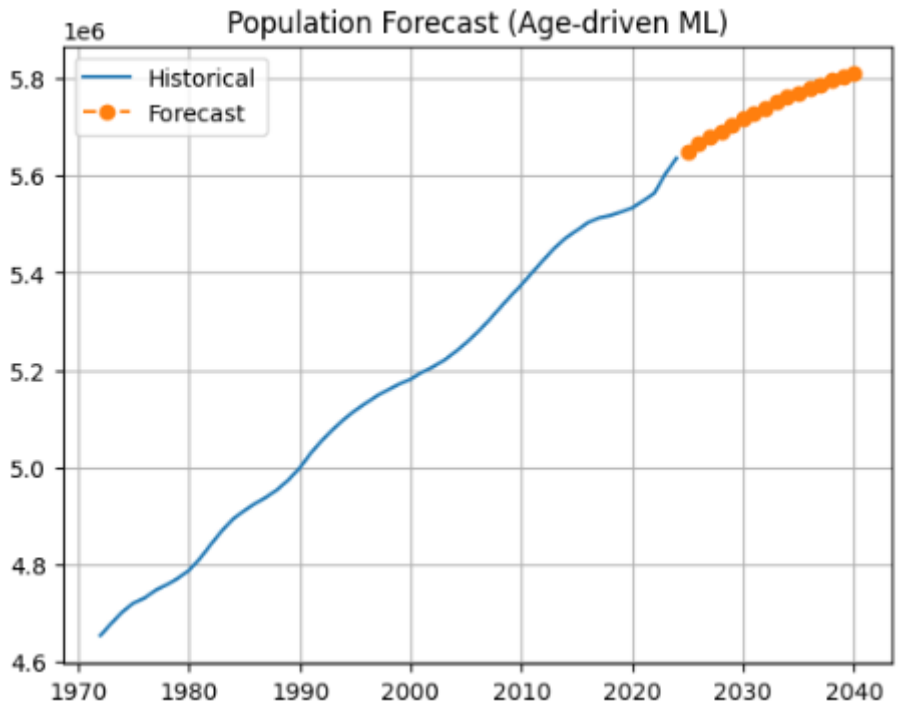




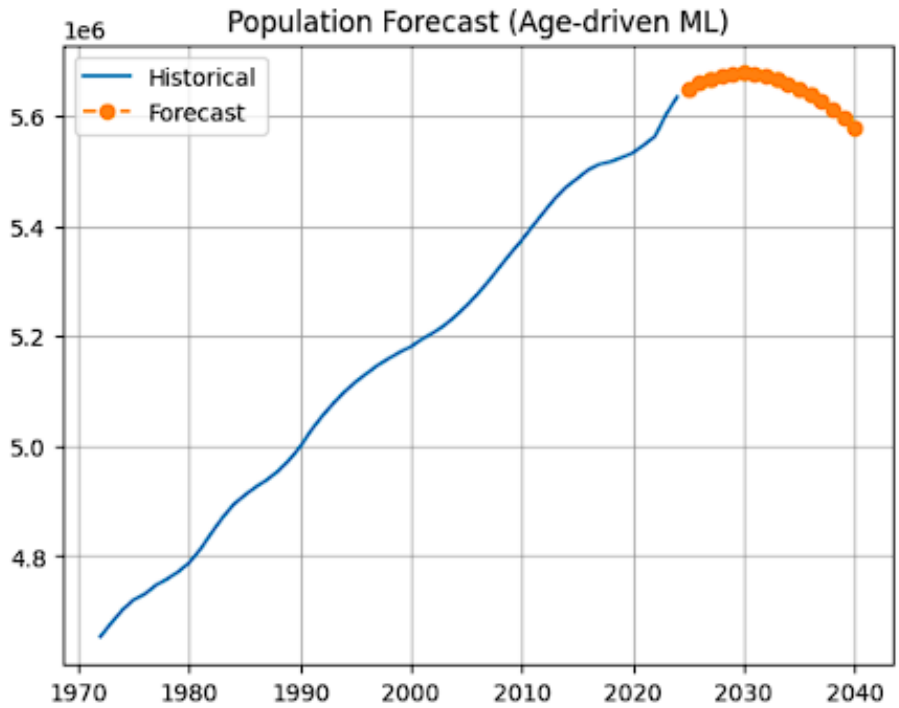
This model hasn't shown big difference, maybe only backrest was more precise. It differes from previous model because it doesn't have dynamic age change.

Then I trained model that use less feature, it doesn't have lag feature. I made it because I wanted to see results when model depends mostly on migration and natural increase. But I have made migration trend base and natural increase is manually counted by births – deaths. This model shown biggest difference:

Supercomputer:



Local machine:



Performance of supercomputer vs local machine:

In my opinion biggest difference is of course speed, all models that I have tried were training at least two times faster.

Next thing is results. I was shocked that they differ, because I wasn't expecting that at all. It could be caused because on supercomputer validation MAE spikes sometimes and on the local machine it becomes more static after 20 epochs. Also, I have trained model with 2000 epoch on Puhti supercomputer and on 200 epochs on local machine. It also can cause difference, even though I had early stopping configured with patience 20 epochs.

Here is log from both:

Supercomputer:

```
Epoch 1/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 486ms/step - loss: 1.2307 - mae: 0.8832 - val_loss: 3.4414 - val_mae: 1.6802
Epoch 2/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.7278 - mae: 0.7158 - val_loss: 3.4610 - val_mae: 1.4000
Epoch 3/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.5433 - mae: 0.6207 - val_loss: 2.7190 - val_mae: 1.1741
Epoch 4/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.3842 - mae: 0.5202 - val_loss: 1.4843 - val_mae: 0.8652
Epoch 5/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.2340 - mae: 0.3880 - val_loss: 0.7615 - val_mae: 0.5592
Epoch 6/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.1293 - mae: 0.2747 - val_loss: 0.9474 - val_mae: 0.7849
Epoch 7/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0959 - mae: 0.2242 - val_loss: 1.1688 - val_mae: 0.9503
Epoch 8/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0996 - mae: 0.2480 - val_loss: 0.9861 - val_mae: 0.9342
Epoch 9/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.1066 - mae: 0.2614 - val_loss: 1.0207 - val_mae: 0.9459
Epoch 10/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0854 - mae: 0.2337 - val_loss: 0.8440 - val_mae: 0.8475
Epoch 11/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0667 - mae: 0.2005 - val_loss: 0.5649 - val_mae: 0.6698
Epoch 12/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0537 - mae: 0.1774 - val_loss: 0.3682 - val_mae: 0.5079
Epoch 13/2000
...
Epoch 82/2000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0143 - mae: 0.0731 - val_loss: 0.2556 - val_mae: 0.4732
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 101ms/step
MAE: 3866.3470865885415
```

Local machine:

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 1s 142ms/step - loss: 1.0904 - mae: 0.8753 - val_loss: 2.8584 - val_mae: 1.5813
Epoch 2/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.8157 - mae: 0.7765 - val_loss: 3.4509 - val_mae: 1.2978
Epoch 3/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.6686 - mae: 0.6970 - val_loss: 3.6589 - val_mae: 1.2761
Epoch 4/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.5215 - mae: 0.6153 - val_loss: 2.2786 - val_mae: 0.9501
Epoch 5/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.3389 - mae: 0.4909 - val_loss: 1.0226 - val_mae: 0.6796
Epoch 6/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.2266 - mae: 0.3857 - val_loss: 0.6034 - val_mae: 0.4815
Epoch 7/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.1303 - mae: 0.2868 - val_loss: 1.0472 - val_mae: 0.8699
Epoch 8/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - loss: 0.1000 - mae: 0.2376 - val_loss: 0.8395 - val_mae: 0.8035
Epoch 9/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0879 - mae: 0.2112 - val_loss: 0.4267 - val_mae: 0.6361
Epoch 10/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.1007 - mae: 0.2492 - val_loss: 0.3967 - val_mae: 0.6007
Epoch 11/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0780 - mae: 0.2143 - val_loss: 0.7795 - val_mae: 0.8044
Epoch 12/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0707 - mae: 0.1938 - val_loss: 0.9220 - val_mae: 0.9136
Epoch 13/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0602 - mae: 0.1959 - val_loss: 0.5918 - val_mae: 0.6383
...
Epoch 70/200
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0153 - mae: 0.0528 - val_loss: 0.1420 - val_mae: 0.3418
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
MAE: 4029.6197916666665
```

As we can see, biggest difference is time which is faster on supercomputer, MAE which is smaller on supercomputer and epoch count which is smaller on local machine.

These values are reason why result on the graph differs too.

Conclusion:

I liked working with supercomputer. I really liked interface of the Puhti supercomputer, it is easily to use. When I got used to it and solved problem with python module, I really enjoyed using it.

Also, I didn't need to wait long in the queue for my session to start, that was nice, since I recreated session about 4 times.

Performance also is on the different level, libraries download fast, model trains way faster. I was curious to see potential of supercomputer, so I have asked AI to build me a heavy model. It somehow had 15 models in the code, and each had 2000 epoch, they were training more than 10 minutes on my local machine and about a minute on supercomputer.

It is hard to see potential of supercomputer with small model as I had. But the heavier, complicated model you build the more potential of supercomputer you can see. That's the conclusion I have come to.

4 References

Grossman, I., Wilson, T., & Temple, J. (2023). Forecasting small area populations with long short-term memory networks. Retrieved from: https://www.sciencedirect.com/science/article/pii/S0038012123001702#bib11